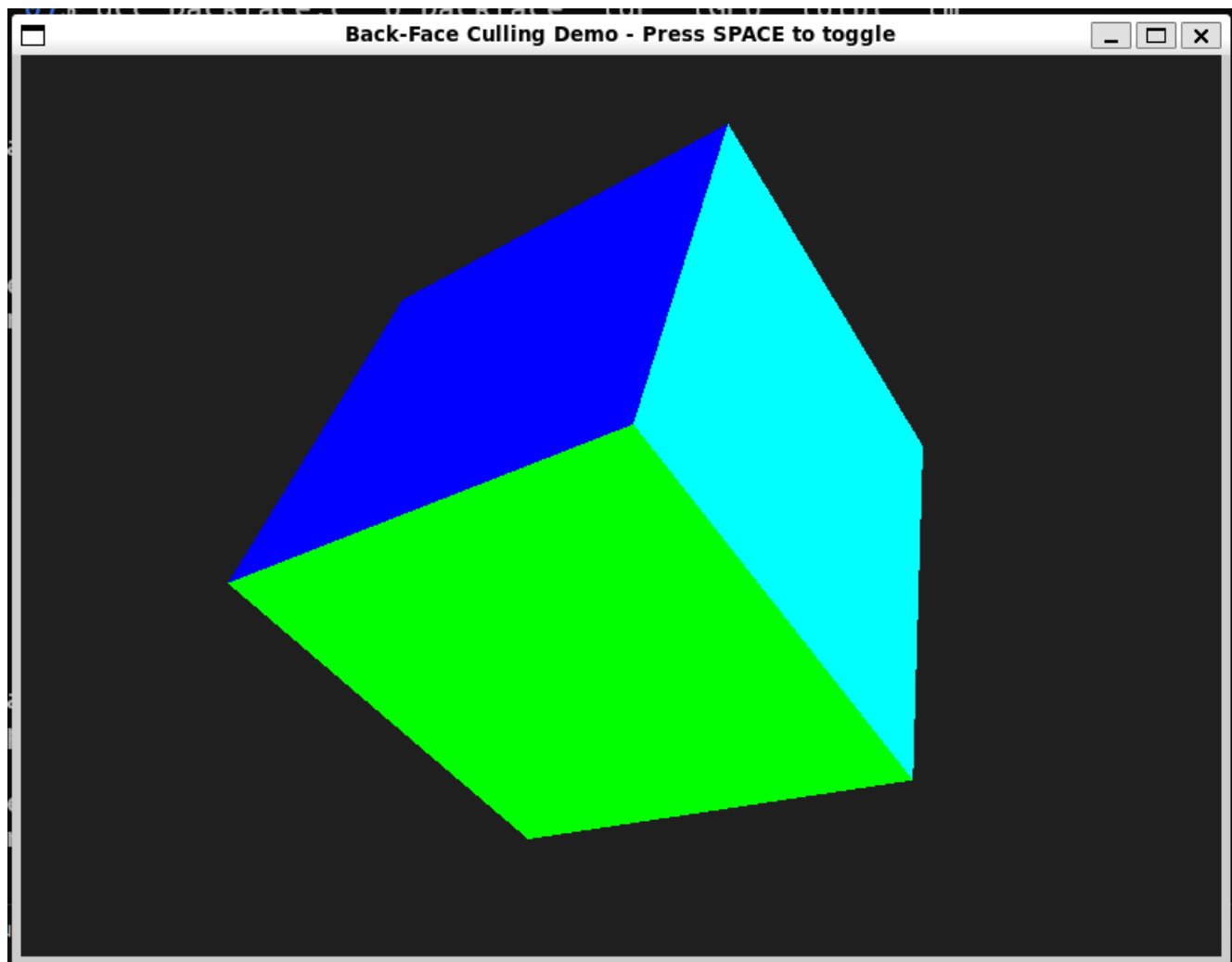


Graphics and Visualization - SE3032

Lab – 07

IT23226128 – D.M.D.G.B.Seneviratne

Back-Face Culling



Code

```
#include <GL/glut.h>  
#include <stdio.h>
```

```
float angle = 0.0f;
```

```
int showBackface = 1; // 1 = show backface, 0 = cull backface
```

```
void drawColoredCube(float size) {
```

```
    float s = size * 0.5f;
```

```
    glBegin(GL_QUADS);
```

```
    // +Z (front) - RED
```

```
    glColor3f(1, 0, 0);
```

```
    glVertex3f(-s, -s, s);
```

```
    glVertex3f(s, -s, s);
```

```
    glVertex3f(s, s, s);
```

```
    glVertex3f(-s, s, s);
```

```
    // -Z (back) - GREEN
```

```
    glColor3f(0, 1, 0);
```

```
    glVertex3f(s, -s, -s);
```

```
    glVertex3f(-s, -s, -s);
```

```
    glVertex3f(-s, s, -s);
```

```
    glVertex3f(s, s, -s);
```

```
    // +Y (top) - BLUE
```

```
    glColor3f(0, 0, 1);
```

```
    glVertex3f(-s, s, s);
```

```
    glVertex3f(s, s, s);
```

```
    glVertex3f(s, s, -s);
```

```
    glVertex3f(-s, s, -s);
```

```
    // -Y (bottom) - YELLOW
```

```
    glColor3f(1, 1, 0);
```

```
    glVertex3f(-s, -s, -s);
```

```
    glVertex3f(s, -s, -s);
```

```
    glVertex3f(s, -s, s);
```

```
    glVertex3f(-s, -s, s);
```

```
    // +X (right) - MAGENTA
```

```
    glColor3f(1, 0, 1);
```

```
    glVertex3f(s, -s, s);
```

```
    glVertex3f(s, -s, -s);
```

```
    glVertex3f(s, s, -s);
```

```

    glVertex3f(s, s, s);
    // -X (left) - CYAN
    glColor3f(0, 1, 1);
    glVertex3f(-s, -s, -s);
    glVertex3f(-s, -s, s);
    glVertex3f(-s, s, s);
    glVertex3f(-s, s, -s);
    glEnd();
}

```

```

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();
    gluLookAt(0, 0, 3, 0, 0, 0, 0, 1, 0);
    glRotatef(40, 0, 0, 1);
    glRotatef(30, 1, 0, 0);
    glRotatef(angle, 0, 1, 0);

    if (showBackface) {
        glDisable(GL_CULL_FACE);
    } else {
        glEnable(GL_CULL_FACE);
        glCullFace(GL_BACK);
    }

    drawColoredCube(1.2f);
    glutSwapBuffers();
}

```

```

void reshape(int w, int h) {
    if (h == 0) h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double)w / (double)h, 0.1, 100.0);
}

```

```

    glMatrixMode(GL_MODELVIEW);
}

void keyboard(unsigned char key, int x, int y) {
    if (key == ' ') {
        showBackface = !showBackface;

        printf("Back-face culling: %s\n", showBackface ? "DISABLED" : "ENABLED");
    }

    glutPostRedisplay();
}

void idle() {
    angle += 0.05f;

    if (angle >= 360.0f) angle -= 360.0f;

    glutPostRedisplay();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);

    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);

    glutInitWindowSize(800, 600);

    glutCreateWindow("Back-Face Culling Demo - Press SPACE to toggle");

    glEnable(GL_DEPTH_TEST);

    glFrontFace(GL_CCW); // set default face winding

    glClearColor(0.12f, 0.12f, 0.12f, 1.0f);

    printf("Press SPACE to toggle back-face culling\n");

    printf("Initial state: Back-face culling DISABLED (you see both sides)\n");

    glutDisplayFunc(display);

    glutReshapeFunc(reshape);

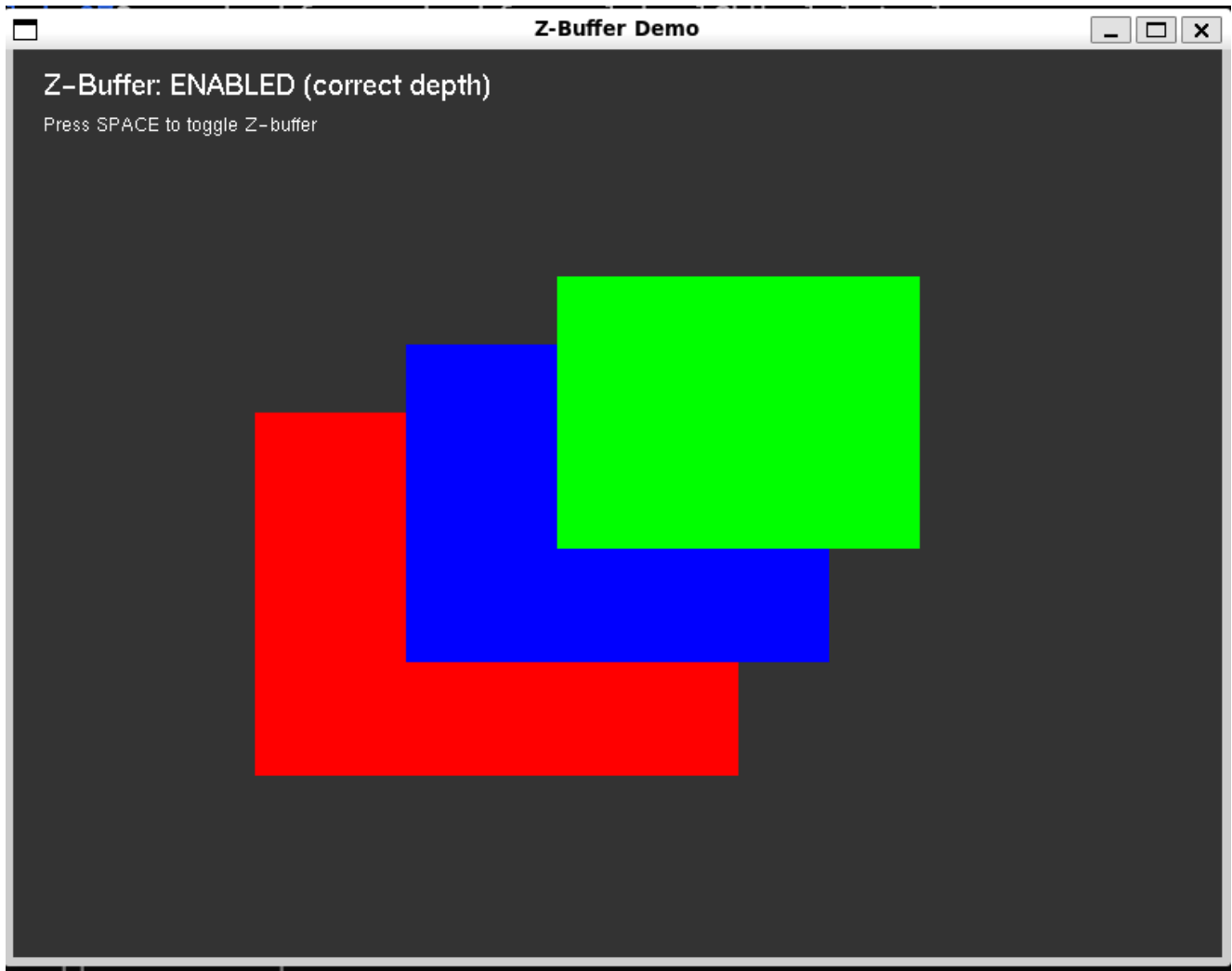
    glutKeyboardFunc(keyboard);

    glutIdleFunc(idle);
}

```

```
    glutMainLoop();  
    return 0;  
}
```

Z-buffer Algorithm in OpenGL



Code

```
#include <GL/glut.h>  
  
#include <stdio.h>  
  
int useZBuffer = 1; // Toggle Z-buffer on/off  
  
void drawSquare(float x, float y, float z, float size, float r, float g, float b) {
```

```

    glColor3f(r, g, b);
    glBegin(GL_QUADS);
    glVertex3f(x - size, y - size, z);
    glVertex3f(x + size, y - size, z);
    glVertex3f(x + size, y + size, z);
    glVertex3f(x - size, y + size, z);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glLoadIdentity();

    // Toggle depth testing based on user input
    if (useZBuffer) {
        glEnable(GL_DEPTH_TEST);
        glDepthFunc(GL_LESS);
    } else {
        glDisable(GL_DEPTH_TEST);
    }

    // Draw in RANDOM order to demonstrate Z-buffer capability
    // Without Z-buffer: render order matters
    // With Z-buffer: Z-values determine visibility

    // Draw GREEN square CLOSE (z = -0.3)
    drawSquare(0.2f, 0.2f, -0.3f, 0.3f, 0.0f, 1.0f, 0.0f);

    // Draw RED square FAR away (z = -0.8)
    drawSquare(-0.2f, -0.2f, -0.8f, 0.4f, 1.0f, 0.0f, 0.0f);

    // Draw BLUE square MIDDLE (z = -0.5)
    drawSquare(0.0f, 0.0f, -0.5f, 0.35f, 0.0f, 0.0f, 1.0f);

    // Display status

```

```

    glDisable(GL_DEPTH_TEST);

    glColor3f(1.0f, 1.0f, 1.0f);

    glRasterPos2f(-0.95f, 0.9f);

    const char* status = useZBuffer ? "Z-Buffer: ENABLED (correct depth)" : "Z-Buffer: DISABLED (render order)";

    for (const char* c = status; *c != '\0'; c++) {

        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_18, *c);

    }

    glRasterPos2f(-0.95f, 0.82f);

    const char* instruction = "Press SPACE to toggle Z-buffer";

    for (const char* c = instruction; *c != '\0'; c++) {

        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, *c);

    }

    glutSwapBuffers();
}

void reshape(int w, int h) {

    glViewport(0, 0, w, h);

    glMatrixMode(GL_PROJECTION);

    glLoadIdentity();

    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);

    glMatrixMode(GL_MODELVIEW);

}

void keyboard(unsigned char key, int x, int y) {

    if (key == ' ') {

        useZBuffer = !useZBuffer;

        printf("Z-Buffer: %s\n", useZBuffer ? "ENABLED" : "DISABLED");

        glutPostRedisplay();

    }

}

int main(int argc, char** argv) {

```

```
glutInit(&argc, argv);

glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);

glutInitWindowSize(800, 600);

glutCreateWindow("Z-Buffer Demo");


glClearColor(0.2f, 0.2f, 0.2f, 1.0f);


glutDisplayFunc(display);

glutReshapeFunc(reshape);

glutKeyboardFunc(keyboard);


printf("Z-Buffer Demonstration\n");

printf("Squares drawn in random order:\n");

// Green (z=-0.3, closest)
// Red (z=-0.8, farthest)
// Blue (z=-0.5, middle)

printf("\nPress SPACE to toggle Z-buffer on/off\n");


printf("\nWith Z-buffer: Correct depth-based visibility\n");

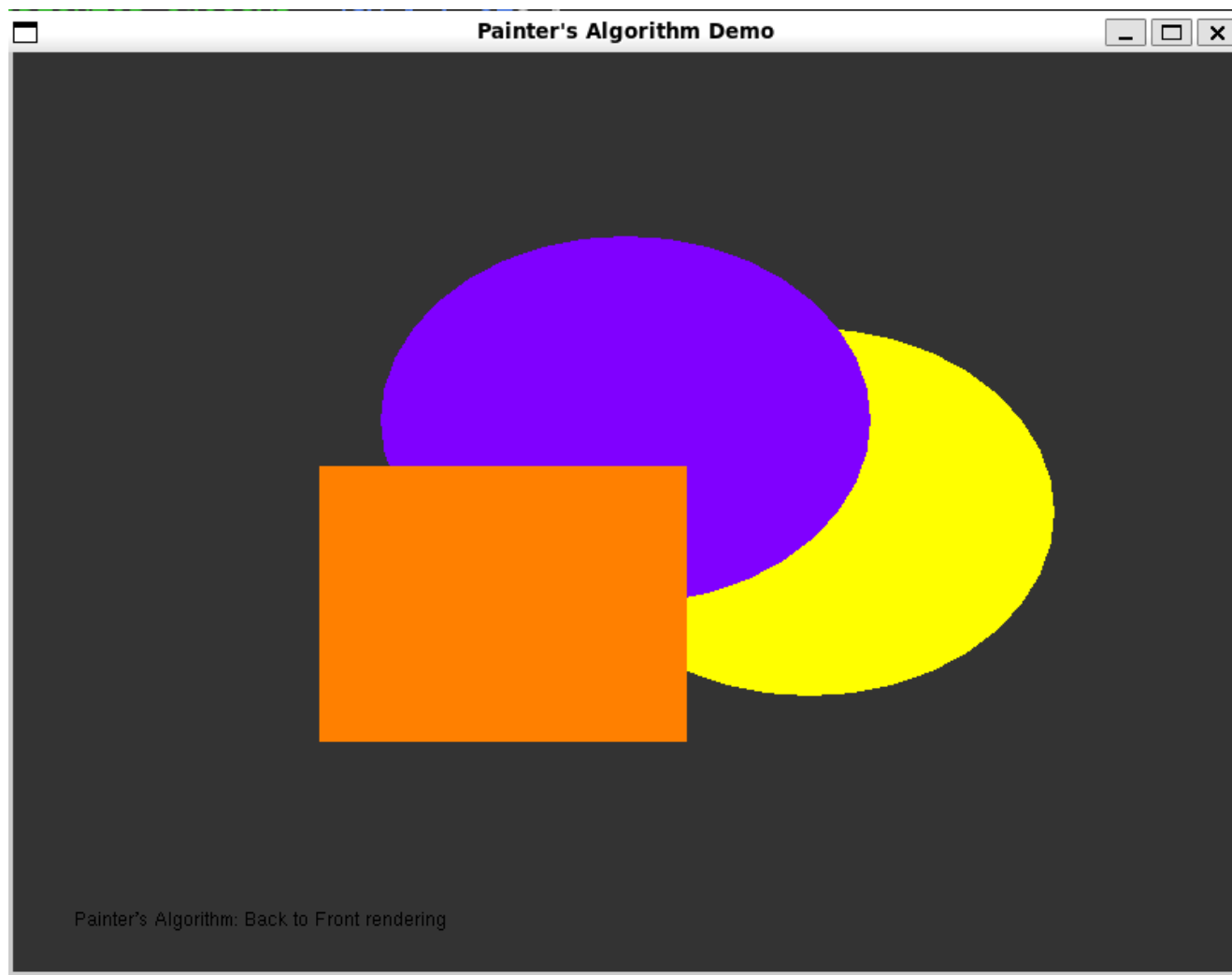
printf("Without Z-buffer: Last drawn appears on top\n");


glutMainLoop();

return 0;

}
```


Depth Sort (Painter's) Algorithm



Code

```
#include <GL/glut.h>
#include <math.h>
#include <stdio.h>

#define PI 3.14159265

void drawCircle(float x, float y, float z, float radius, float r, float g, float b) {
    glColor3f(r, g, b);
    glBegin(GL_POLYGON);
    for (int i = 0; i < 360; i += 10) {
```

```

        float angle = i * PI / 180.0f;

        glVertex3f(x + radius * cos(angle), y + radius * sin(angle), z);
    }
    glEnd();
}

void drawSquare(float x, float y, float z, float size, float r, float g, float b) {
    glColor3f(r, g, b);
    glBegin(GL_POLYGON);
    glVertex3f(x - size, y - size, z);
    glVertex3f(x + size, y - size, z);
    glVertex3f(x + size, y + size, z);
    glVertex3f(x - size, y + size, z);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glLoadIdentity();

    glDisable(GL_DEPTH_TEST);

    drawCircle(0.3f, 0.0f, -0.5f, 0.4f, 1.0f, 1.0f, 0.0f); // Yellow circle
    drawCircle(0.0f, 0.2f, -0.3f, 0.4f, 0.5f, 0.0f, 1.0f); // Purple circle
    drawSquare(-0.2f, -0.2f, -0.1f, 0.3f, 1.0f, 0.5f, 0.0f); // Orange square

    glColor3f(0.0f, 0.0f, 0.0f); // Black text
    glRasterPos2f(-0.9f, -0.9f);
    const char* text = "Painter's Algorithm: Back to Front rendering";
    for (const char* c = text; *c != '\0'; c++) {
        glutBitmapCharacter(GLUT_BITMAP_HELVETICA_12, *c);
    }

    glutSwapBuffers();
}

```

```
void reshape(int w, int h) {  
    glViewport(0, 0, w, h);  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);  
    glMatrixMode(GL_MODELVIEW);  
}  
  
int main(int argc, char** argv) {  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);  
    glutInitWindowSize(800, 600);  
    glutCreateWindow("Painter's Algorithm Demo");  
  
    glClearColor(0.2f, 0.2f, 0.2f, 1.0f);  
  
    glutDisplayFunc(display);  
    glutReshapeFunc(reshape);  
  
    // It draws stuff from back to front: Yellow circle (farthest), Purple circle (middle), Orange square (closest)  
  
    glutMainLoop();  
    return 0;  
}
```